

分散学習クイズアプリ(SPA) 要件定義・基本設計書

1. プロジェクト概要

忘却曲線(SM-2アルゴリズムベース)に沿って最適な復習タイミングを算出し、学習効率を最大化するためのWebベースのクイズアプリ。

ユーザーが作成したCSVファイルのインポートに対応し、学習記録を保持したまま問題のアップデートが可能な「スマートマージ機能」を備える。

2. 技術スタックとアーキテクチャ

- フロントエンド: HTML5, CSS3, JavaScript (Vanilla JS / ES6+)
 - フレームワーク非依存で軽量に構築。将来的にはPWA化やCapacitor等でのネイティブアプリ化を見据える。
- アーキテクチャ: SPA(Single Page Application)
 - 画面遷移(リロード)を行わず、JavaScriptによるDOM操作でビュー(ダッシュボード・問題・解説)を高速に切り替える。
- データ保存: `localStorage` (JSON形式)
 - 将来的にはGoogle Drive APIを利用したクラウド同期へ拡張予定。

3. ディレクトリ・ファイル構成

Plaintext

```
/quiz-app
├── index.html    (メインUI、各Viewのコンテナ)
├── css/
│   └── style.css (スタイリング。選択枝の左寄せや選択状態のUI等を含む)
├── js/
│   └── main.js   (状態管理、CSVパース、SM-2ロジック、DOM操作)
└── images/      (問題・解説用の画像格納ディレクトリ)
```

4. データ設計(ストレージとCSVマージ)

4-1. アプリ内データ(JSON)

`localStorage` 内で `myQuizData` というキーで以下の構造を保存する。静的な問題データと動的な学習データを統合して管理する。

- 静的データ: id, question, correctAnswers (配列), options (配列), explanation, questionImage, explanationImage
- 学習データ: repetitionCount (連続正解数), easeFactor (定着度), intervalDays (次回までの日数), nextReviewDate (次回出題タイムスタンプ)

4-2. CSVインポートとスマートマージ

ユーザーがアップロードするCSV(全12列構成)を読み込み、内部でJSONに変換する。

- プレフィックスの自動カット: 読み込み時、正規表現を用いて問題文や選択肢の先頭にある「Q1.」や「1.」等を自動削除する。
- ID照合マージ: CSVの ID 列を主キーとする。
 - 既存IDの場合: 学習データ(easeFactor等)は上書きせず保護し、問題文・選択肢・解説・画像パスなどの静的データのみを最新に更新する。
 - 新規IDの場合: 新しいオブジェクトとして配列に追加し、学習データは初期値(未学習状態)をセットする。

5. UI/UX設計と画面遷移

画面は大きく3つのViewで構成され、CSSの display: none / block で切り替える。

① ダッシュボード画面 (dashboard-view)

- 統計表示: 全体の着手率、本日のタスク(新規・復習)数、要注意(間違えた問題)数を表示。
- 学習開始: 「通常の学習」と「間違えた問題の復習」の2つのキューを分けて生成し、別々のボタンから開始できる。
- インポート: CSVファイルを選択してアップロードするインターフェース。

② クイズ(問題)画面 (quiz-view)

- 出題・選択肢のランダム化: 出題順(キュー)および、各問題の選択肢の並び順は出題のたびにシャッフルされる。
- UIの最適化: 長文テキストでも認知負荷を下げるため、選択肢ボタン内のテキストは「左寄せ+左パディング」とする。
- 複数選択の自動判定:
 - 正解が複数の場合、問題文末尾に「(※ Xつ選んでください)」を自動付与。
 - 選択したボタンは視覚的にハイライト(トグル可能)される。
 - 必要な選択数(X個)に達した瞬間に、自動で答え合わせ画面へ遷移する(「決定」ボタンを省くUX)。

③ 結果・解説画面 (result-view)

- 正誤判定、問題文、正解の選択肢テキスト、解説文、および画像をシームレスに表示。
- 正解時: 「難しかった」「普通」「簡単」の3ボタンを表示。各ボタンには次回出題タイミングの予測(例: 12時間後、4日後など)をリアルタイム計算して表示する。
- 不正解時: 評価ボタンを隠し、「解説を読んで次へ進む」の単一ボタンを表示。押下時に自動で要注意リスト(キュー)へ送る処理を行う。

6. 学習アルゴリズム(SM-2改)

SuperMemo-2(SM-2)アルゴリズムをベースに、分・時間単位の細かいスケジュール管理を追加。

- 正解時: ユーザーの評価(難・普・易)に応じて `easeFactor` を変動させ、次のインターバルを算出($\text{intervalDays} = \text{現在のインターバル} * \text{easeFactor} * \text{評価ボーナス}$)。
- 不正解時: `repetitionCount` を0にリセットし、`intervalDays` を0(要注意キューへ移動)とする。`easeFactor` はペナルティとして減少(最低値1.3)。
- 表示フォーマット: インターバルの日数(少数)を計算し、「10分後」「12時間後」「X日後」「Xヶ月後」と人間が直感的に理解できるフォーマットに変換してUIに表示する。

7. 今後の開発計画(ロードマップ)

本アプリは、まずはローカル環境でサクサク動く「コア機能の完成」を第一段階としていますが、将来的にはさらに利便性を高めるため、以下の拡張を段階的に実装していく予定です。

フェーズ1: クラウド同期機能(マルチデバイス対応)

現在の `localStorage` 保存では「同じブラウザ」でしか学習データを引き継げないため、**Google Drive API** を導入し、学習データ(JSON)をクラウド上に保存・同期する仕組みを構築します。

- メリット: PCで追加した問題や学習した続きを、通勤中にスマートフォンのブラウザでそのまま再開できるようになります。キャッシュクリアによるデータ消失のリスクもなくなります。

フェーズ2: PWA(Progressive Web App)化

現在のWebアプリを、スマートフォンの「ホーム画面に追加」して、ネイティブアプリのように単独のアイコンから起動できるようにします。

- **Service Worker**の導入: オフライン状態でもアプリが起動し、ネットワークがない環境(地下鉄など)でも学習を進められるようにします。

フェーズ3: 高度な学習機能と可視化

- 学習グラフの導入: ダッシュボードに「過去7日間の学習時間・問題数」や「記憶定着率」を折れ線グラフ(Chart.js等)で可視化し、モチベーションの維持を図ります。

- タグ・カテゴリ機能の追加: CSVに「カテゴリ(例:循環器、法規など)」の列を追加し、「今日は〇〇の分野だけ復習する」といった絞り込み学習を可能にします。
- UIアニメーション: 正解・不正解時の視覚的なフィードバック(エフェクト)や、画面切り替え時のスムーズなトランジションを追加し、手触り感を向上させます。